Phase 2: Backend Development & Configuration

System Architecture & Guided Implementation Guide

Activity 1: Creation of Custom Table to Store Ticket Records

This milestone establishes the database architecture foundation by spinning up a dedicated custom tracking schema table within the system environment platform.

Technical Navigation & Action Steps:

1. **Locate System Tables:** Open the primary **All** application menus and search for **Tables** in the application filter navigator.
2. **Access System Registry:** Under the **System Definition** dropdown module, click on **Tables** to launch the active schema registry.
3. **Initialize New Configuration:** Click on the **New** action button located on the top right-hand corner toolbar of the registry list view.
4. **Define Schema Metadata:** In the table definition form layout workspace, configure the following primary parameter attributes:
   • **Label:** Incident WorkFlow
   • **Create module:** Uncheck this checkbox option *(to suppress default panel entry and retain clean visual application workspaces)*.
5. **Commit Initial Form State:** Click the **Save / Submit** button on the header control action layout to record the initial schema state.
6. **Configure Auto-Number Engine:** Once the form reloads, scroll downward or click into the dedicated **Controls** section block. Locate the **Auto Number** parameter and explicitly enable the checkbox to set its state value to **True**.
7. **Commit Engine Settings:** Click the form **Save** action block a second time to safely commit the automated tracking baseline configuration rules.
8. **Launch Layout Designer Canvas:** Look underneath the form base workspace block and navigate through the active **Related Links** cluster. Click on the **Show Form** link layout view. From the loaded test record view form layer, right-click the contextual frame layout header, open the **Configure** submenu link tree, and select **Form Designer**.

Activity 2: Field Creation and Data Type Configuration

The tactical goal of this implementation module is to systematically construct all the requisite core tracking attributes directly inside the custom form designer canvas environment, enforcing reliable data consistency and uniform type mapping schema constraints.

Field Label	Data Type	Target Reference / Choice Definition Array
Number	Auto Number	System Autogenerated Tracking Sequence Node Mapping
Caller	Reference	sys_user (Core System Profile Table Link)
Category	Choice	[Network, Hardware, Access, Performance]
Subcategory	Choice	[Wi-Fi, Projector, Forgot Password, Slow Computer]
Short Description	String	Free Text Input String (Short Span)
Description	String	Free Text Input Area Block (Paragraph Span)
State	Choice	[New, In progress, On hold, Resolved, Closed]
Assigned Group	Reference	sys_user_group (Core System Support Structure Matrix)
Assigned to	Reference	sys_user (Active System Full-Fulfillment Operator Profiles)

Post-Processing Choice List Population Instructions:

1. Once fields are arranged within the designer grid layout canvas framework, click into the explicit **Configuration Action Gear Icon Indicator** positioned at the right-edge node boundary of the following Choice List fields: **Category**, **Subcategory**, and **State**.
2. Interactively expand and add the exact selection option values mapped out explicitly in the tabular structure reference definition row layout overview.
3. After configuring choices for **Category**, **Subcategory**, and the workflow **State** parameters, click the primary **Save** layout commit control button located at the upper right corner boundary area node of the designer tool dashboard.
4. Return into the standard browser instance view framework, execute a clean **Reload Form / Page refresh action sequence**, and confirm the data dictionary schema components parse successfully.

Activity 3: Implementing Dependency Between Category and Subcategory Choice Fields

This implementation section handles the integration of conditional execution paths between elements so that selection variants under the Subcategory drop-down filter conditionally based on values assigned to the primary Category field node entry.

Advanced Dictionary Dependency Configuration Steps:

1. **Access Field Elements:** Open the freshly instantiated custom layout form view, hover over the active **Subcategory** input label, and perform a mouse right-click context selection sequence.
2. **Launch Configuration Dictionary:** Select the **Configure Dictionary** element entry option line from the pop-up context option dropdown tree.

3. **Toggle Deep Settings:** At the base base area interface block of the loaded dictionary entry profile layout page, seek out the **Related Links** utility links box and click specifically on **Advanced view**.
4. **Establish Dependent Choice Rules:** Scroll mid-way through the advanced parameters overview layout panel down to the dedicated **Dependent Field** section panel block, and carry out these explicit actions:
   • Enable the operational setting checkbox labeled **Use dependent field** to set its Boolean value to **True**.
   • Use the lookup selector tool node inside the **Dependent on field** configuration line entry to look up and choose the **Category** column layout field record node.
5. **Save Active Form State:** Click on the form header action panel zone **Save** execution asset choice.

Dependent Choice Hierarchical Architecture Mapping Matrix:

Scroll down directly to the live **Choices** interactive records database list table at the base of the framework form and update the relative relational backend key definitions exactly as mapped below:

Subcategory Label Option	Operator Arrow	Parent Category Value Link Map
Wi-Fi	→	**Network**
Projector	→	**Hardware**
Forgot password	→	**Access**
Slow computer	→	**Performance**

Implementation Note: Save or Update the master form container. Open an active ticket sandbox testing form to verify that altering the primary Category option filters and updates the dependent Subcategory values instantaneously.